

Rapport d'Avancement PFA

Federated Learning pour la Détection d'Attaques Active Directory

Phases 1 → 4 : Data, ML, FL + SecAgg, Attaques & Défenses

El Mehdi Boussalem

Aymane Choukry

Mohammed Benlafkih

Mai 2026

nord.corp | DC01 : 192.168.0.47 | Clients : .41/.42/.43

1 Phase 1 — Préparation des Données

Dataset : OTRF/Mordor — **826 000 événements** Windows (Sysmon + Security logs), couvrant 9 classes d'attaques AD réelles (APT29, Kerberoasting, Pass-the-Hash, SAM Dump, etc.).

Feature Engineering : 29 features extraites (event counts, timing, process, network) + StandardScaler.

Partition Non-IID réaliste

- **CLIENT01** : classes [0,1,2,3] — 64 échantillons
 - **CLIENT02** : classes [0,4,5,6] — 68 échantillons
 - **CLIENT03** : classes [0,7,8] — 49 échantillons
- Chaque client voit uniquement un sous-ensemble des attaques.

2 Phase 2 — Modèle ML Baseline

Architecture MLP : Input(29) → 64 → ReLU → Drop(0.2) → 32 → ReLU → Drop(0.2) → 9 (4 297 paramètres).

F1 macro centralisé : **~0.61**. Libraries : PyTorch, NumPy, Pandas, Scikit-learn.

3 Phase 3 — Federated Learning Sécurisé

3.1 Infrastructure Active Directory

Machine	Rôle	IP	OS
DC01	DC principal + AD CS + NPS RADIUS	192.168.0.47	Server 2022
DC02	DC backup (réplication)	—	Server 2022
CLIENT01	Client FL #1 (attacker data)	192.168.0.41	Windows 10
CLIENT02	Client FL #2 (victim data)	192.168.0.42	Windows 10
CLIENT03	Client FL #3 (dump data)	192.168.0.43	Windows 10
FL Server	Flower + Auth Server	192.168.0.1	Ubuntu

TABLE 1 – Infrastructure du projet

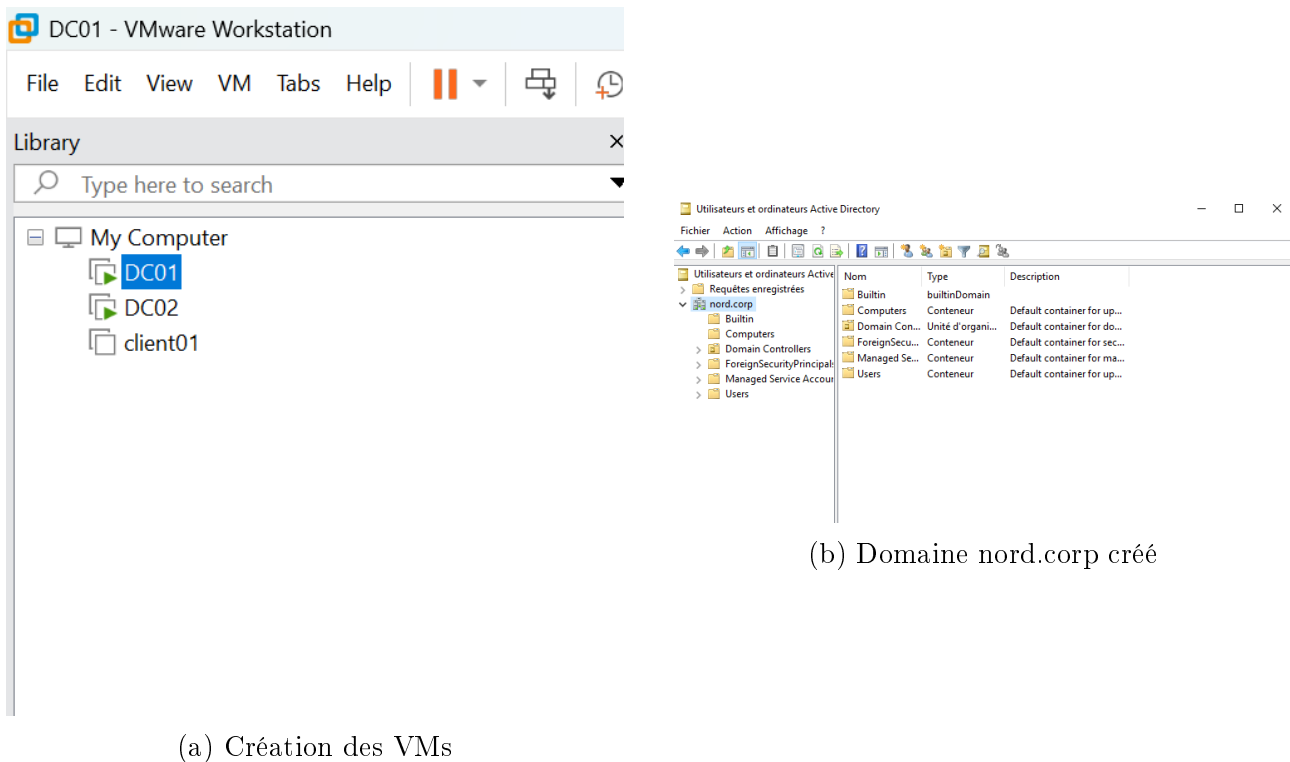
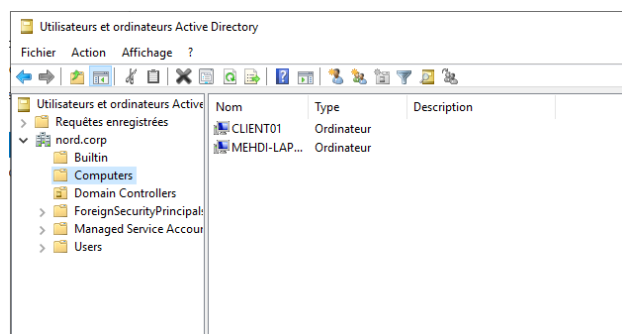
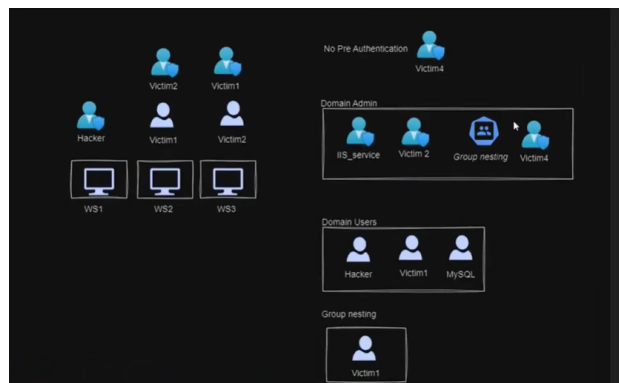


FIGURE 1 – Mise en place de l'infrastructure VMware



(a) Machines du domaine

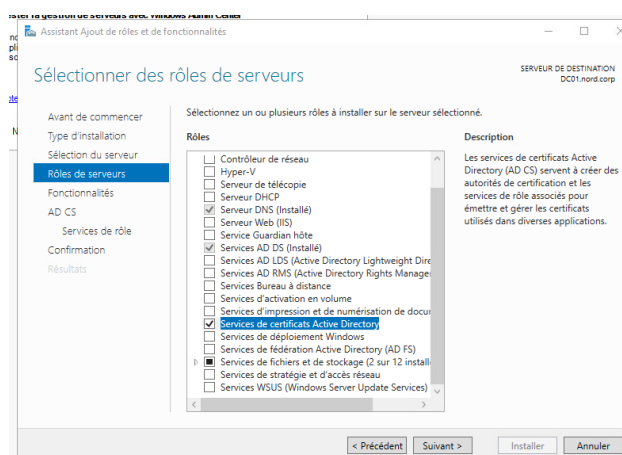


(b) Arborescence utilisateurs AD

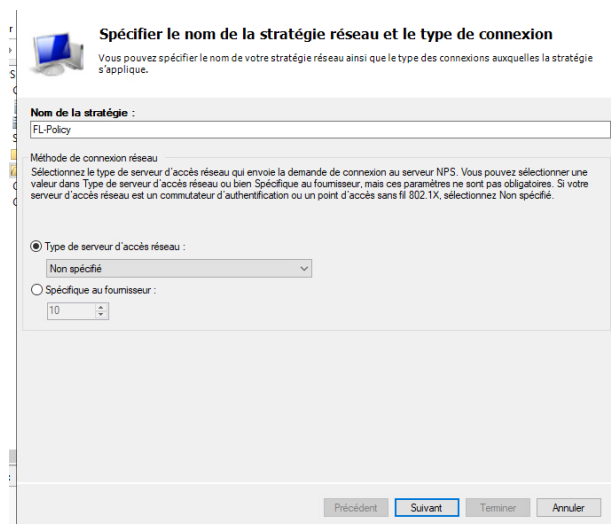
FIGURE 2 – Active Directory — machines et utilisateurs

3.2 PKI (AD CS) + RADIUS (NPS)

AD Certificate Services (FL-Root-CA) émet les certificats mTLS. NPS RADIUS authentifie les clients FL.



(a) Installation AD CS



(b) Network Policy NPS

FIGURE 3 – PKI et RADIUS — sécurisation de l'authentification FL

Pipeline d'authentification

Client $\xrightarrow{\text{mTLS}}$ Auth Server (8443) $\xrightarrow{\text{RADIUS PAP}}$ NPS/DC01 (1812) $\rightarrow$ token HMAC-SHA256 $\rightarrow$ Flower gRPC (8444)

3.3 Entraînement Fédéré (Flower)

Paramètre	Valeur	Paramètre	Valeur
Framework	Flower 1.13	Local Epochs	3
Stratégie	FedAvg	Learning Rate	0.001
Rondes	30	Communication	gRPC + mTLS

TABLE 2 – Configuration de l’entraînement FL

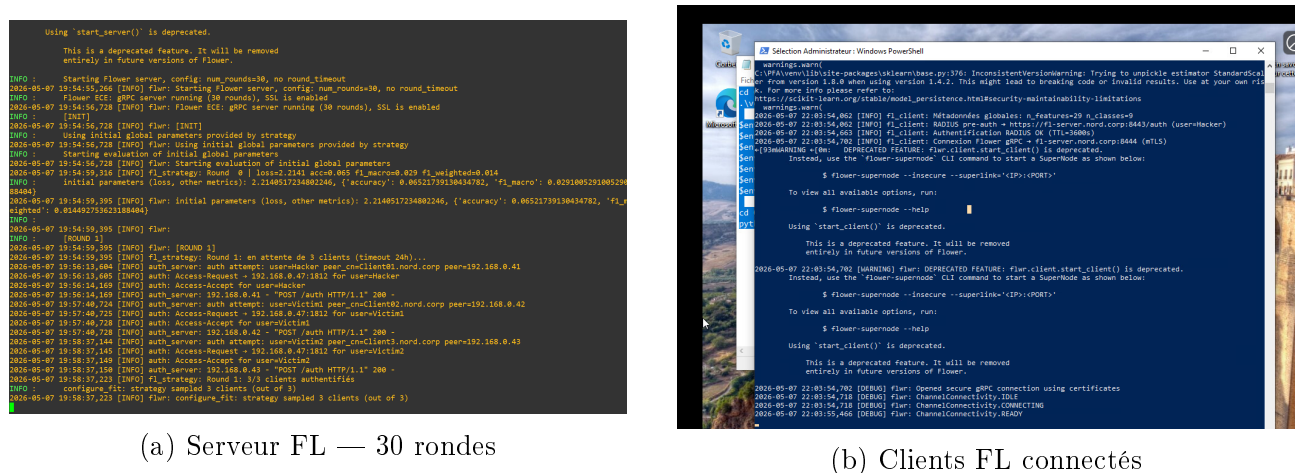


FIGURE 4 – Lancement de l'entraînement fédéré



FIGURE 5 – Résultats et déploiement

Mode	F1 Macro	Note
Centralisé (baseline)	~0.61	Toutes données combinées
FL Vanilla (FedAvg)	~0.27	Données non-IID → dégradation attendue
FL + SecAgg+	~0.18	Coût de la confidentialité

TABLE 3 – Comparaison des performances

4 Phase 4 — Attaques sur le FL

4.1 Attaque #1 — Serveur Curieux (Honest-but-Curious)

Principe : Le serveur intercepte les paramètres bruts de chaque client à chaque ronde (30 fichiers .npz). Distance L2 entre clients $\neq 0 \rightarrow$ le serveur **distingue** les clients. **Vulnérabilité confirmée**.

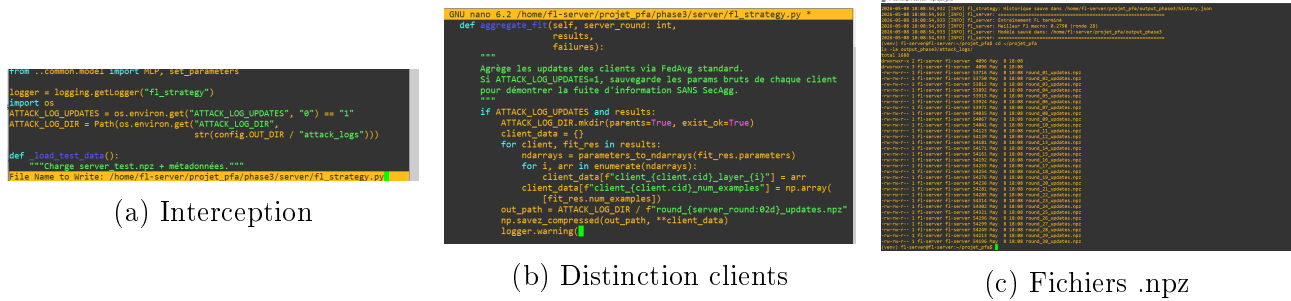


FIGURE 6 – Attaque #1 — Serveur curieux intercepte les paramètres individuels

4.2 Attaque #2 — Property Inference

Principe : Analyse de la déviation des poids fc3 pour inférer les classes de données par client.

Client	Classes inférées	Classes réelles	Précision
CLIENT01	[1,2,3,5]	[0,1,2,3]	75%
CLIENT02	[4,5,6,7]	[0,4,5,6]	75%
CLIENT03	[4,5,7,8]	[0,7,8]	67%

TABLE 4 – Résultats de l'attaque Property Inference

ft-server@ft-server: ~/projet_pfa

[+] 3 clients detectes

Tableau des scores de deviation par (client, classe):
(score eleve = classe probablement presente dans le dataset du client)

Client	cls_0	cls_1	cls_2	cls_3	cls_4	cls_5	cls_6	cls_7	cls_8
0be6bfe92ac4a	0.021	0.017	0.016	0.015	0.024	0.025	0.023	0.052	0.052
5a8284dfb2b4e	0.021	0.049	0.048	0.046	0.024	0.025	0.023	0.017	0.018
c4e0fdd65a994c	0.025	0.033	0.033	0.031	0.048	0.050	0.046	0.035	0.034

=====

INFERENCE - top-4 classes les plus probables par client

=====

Client 0be6bfe92ac4a -> classes [4, 5, 7, 8]
scores correspondants: ['0.052', '0.052', '0.025', '0.024']

Client 5a8284dfb2b4e -> classes [1, 2, 3, 5]
scores correspondants: ['0.049', '0.048', '0.046', '0.025']

Client c4e0fdd65a994c -> classes [4, 5, 6, 7]
scores correspondants: ['0.050', '0.048', '0.046', '0.035']

FIGURE 7 – Property Inference — résultats d'analyse

Verdict : L'attaquant infère les données de chaque client avec 67-75% de précision $\rightarrow$ les paramètres bruts **fuient de l'information**.

4.3 Mitigation — SecAgg+ (Secure Aggregation Plus)

Principe mathématique de SecAgg+

Chaque paire (i, j) calcule un secret partagé via X25519 Diffie-Hellman :

$$\text{seed}_{ij} = \text{HKDF-SHA256}(\text{X25519}(sk_i, pk_j))$$

Avant envoi, chaque client masque ses paramètres :

$$\text{masked}_i = \text{params}_i + \sum_{j>i} \text{mask}_{ij} - \sum_{j<i} \text{mask}_{ji}$$

À l'agrégation, les masques s'annulent :

$$\sum_i \text{masked}_i = \sum_i \text{params}_i$$

→ Le serveur obtient la **somme** sans voir les paramètres individuels.

4.3.1 Implémentation — 4 Patches

Patch	Fichier	Rôle
1/4	secagg.py	Crypto : X25519 key gen, HKDF seed, mask generation
2/4	auth_server.py	Routes /pubkey et /pubkeys pour échange de clés
3/4	fl_client.py	Génération clés X25519, masquage params avant envoi
4/4	fl_strategy.py	Agrégation somme masquée (sans accès individuel)

TABLE 5 – Les 4 patches SecAgg+

4.3.2 Stack Cryptographique

Composant	Technologie	Rôle
Échange de clés	X25519 (Curve25519)	Secret partagé pairwise
Dérivation	HKDF-SHA256	Seed → masque déterministe
Masquage	PRG (numpy.random)	Génération masques $\pm$
Transport	mTLS + gRPC	Protection en transit

TABLE 6 – Stack cryptographique SecAgg+

4.3.3 Résultats Avant / Après SecAgg+

Métrique	Sans SecAgg	Avec SecAgg+
Attaque #1 (Serveur Curieux)	Paramètres individuels visibles	Somme masquée uniquement
Attaque #2 (Property Inference)	67-75% de précision	~11% (aléatoire)
F1 Macro	~0.27	~0.18
Confidentialité	Aucune	Garantie (masquage pair-wise)

TABLE 7 – Impact de SecAgg+ — tableau comparatif

5 Architecture Globale & Discussion

5.1 Architecture du Système

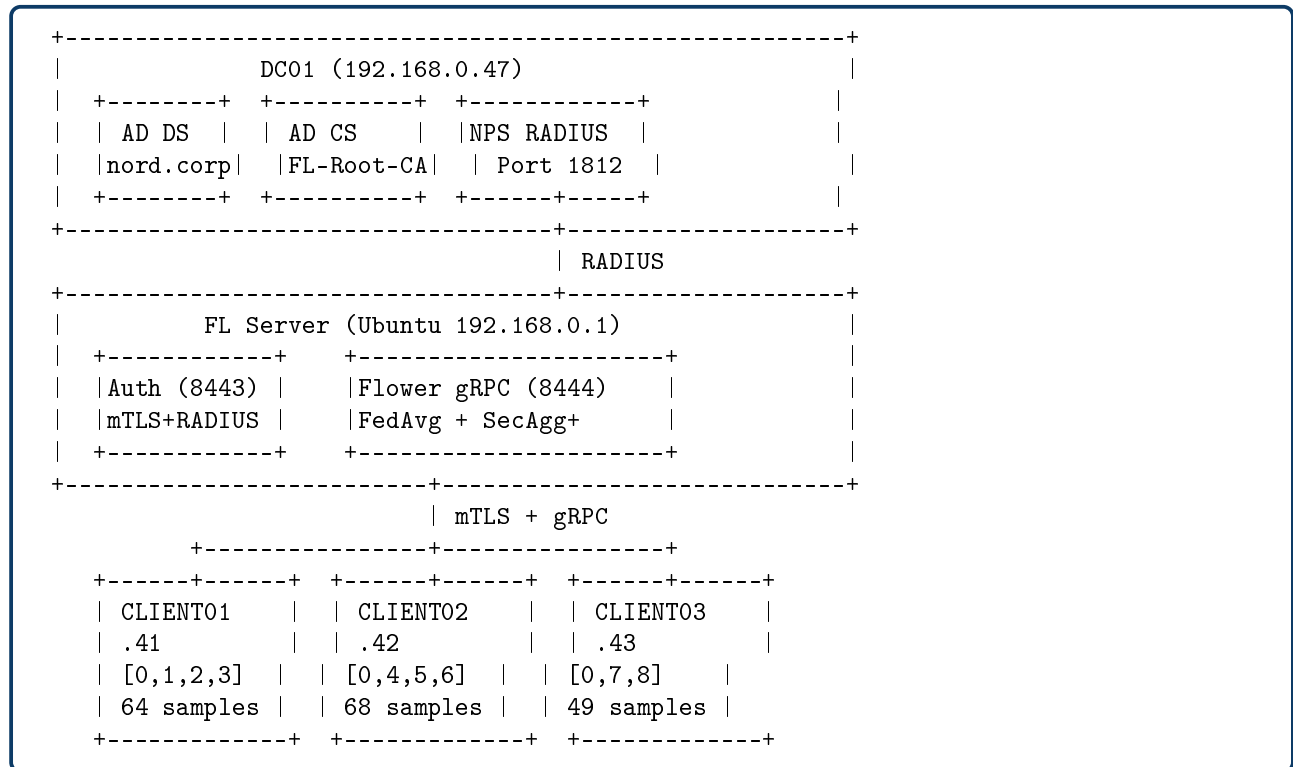


FIGURE 8 – Architecture globale du systeme FL securise

5.2 Discussion

Confidentialité vs Performance : SecAgg+ protège les paramètres individuels au prix d’une dégradation F1 (0.27→0.18). Ce compromis est acceptable car le but premier est la protection de la vie privée des données d’entraînement.

Limites identifiées :

- **Intégrité :** SecAgg+ ne protège pas contre un client malveillant (Free-Rider) → Robust Aggregation nécessaire (Krum, Trimmed Mean)
- **Non-IID :** Distribution déséquilibrée → optimisation hyperparamètres (LR=0.0005, LOCAL_EPOCHS=1)
- **Scalabilité :** $O(n^2)$ échanges de clés — acceptable pour 3 clients, coûteux à grande échelle

5.3 Prochaines Étapes

#	Tâche	État
1	Optimisation hyperparamètres FL	Planifié
2	Agent détection temps réel (<code>realtime.py</code>)	Planifié
3	Attaques AD (Kerberoasting, PtH, LLMNR, BloodHound)	En cours
4	Hardening AD + validation post-mitigation	Planifié
5	Rapport final LaTeX complet	En cours

TABLE 8 – Plan des prochaines étapes